Week 15: How Does a Neural Network Learn?

Finish the MLP, Evaluate a Classifier, and Derive Backpropagation by Hand

Instructor / Mentor

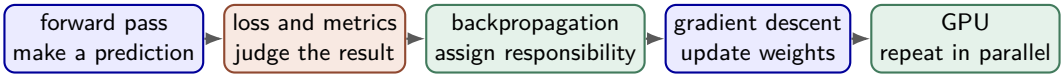
Kiki Research Training Program

Week 15

Contents

Reconnect: Prediction, MLPs, and Evaluation

Today's Story: Predict, Judge, Learn, Accelerate



Central question

If a network's prediction is wrong, how does every weight learn how much it contributed to that error?

Learning Objectives

By the end of class, students can:

1. complete a forward pass through a small MLP;
2. calculate a confusion matrix, precision, recall, and F1-score;
3. distinguish a per-example loss from a test-set evaluation metric;
4. explain a gradient as "which direction changes the loss";
5. derive one complete backward pass using the chain rule;
6. update all weights with gradient descent;
7. explain why matrix calculations fit a GPU; and
8. explain why training usually needs more GPU memory than inference.

120-Minute Flow

1. **0–20 min:** finish the MLP forward pass and function view.
2. **20–38 min:** classifier evaluation and threshold decisions.
3. **38–53 min:** loss functions and gradient-descent intuition.
4. **53–58 min:** break.
5. **58–91 min:** derive backpropagation and update a tiny network by hand.
6. **91–108 min:** vectorization, CPU versus GPU, and memory needs.
7. **108–116 min:** interactive network-function demonstration.
8. **116–120 min:** exit ticket and homework briefing.

Three Different Questions

Quantity	Question	Example
Prediction	What does the model output now?	$\hat{p} = 0.63$ useful
Loss	How wrong is this prediction for training?	binary cross-entropy $= 0.47$
Metric	How useful are decisions on unseen data?	recall $= 0.80$, F1 $= 0.76$

Do not mix their jobs

The loss sends a smooth learning signal to the weights. Metrics communicate real-world performance and may contain hard thresholds.

Finish the Multilayer Perceptron

A Neuron Has Two Steps

1. Combine information

$$z = w_1 x_1 + w_2 x_2 + \cdots + b$$

The weights decide the importance and direction of each input.

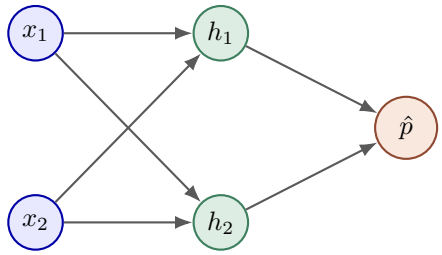
2. Apply an activation

$$a = g(z)$$

The activation changes the shape the network can represent.

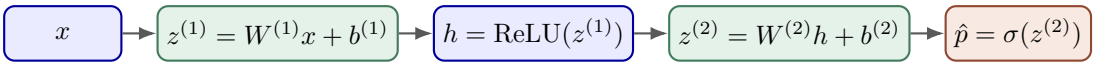
Logistic regression is one sigmoid neuron. An MLP connects many neurons in layers.

A Small MLP for Two Input Features



$$z^{(1)} = W^{(1)}x + b^{(1)}, \quad h = \mathrm{ReLU}(z^{(1)}), \quad z^{(2)} = W^{(2)}h + b^{(2)}, \quad \hat{p} = \sigma(z^{(2)}).$$

The Forward Pass Is a Chain of Functions

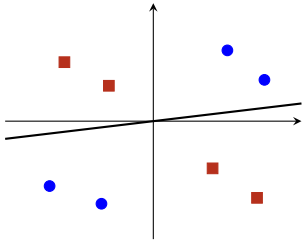


Forward means left to right

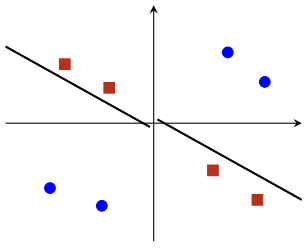
Each layer transforms the representation. Training will send responsibility in the reverse direction.

Why Hidden Layers Matter: Bend the Boundary

one neuron



hidden neurons combine regions



The Function-Approximation View

Supervised learning

We observe example pairs (x, y) and build a function f_θ so that

$$f_\theta(x) \approx y.$$

The symbol θ means all trainable weights and biases.

- One input dimension: the network draws a curve $\hat{y} = f_\theta(x)$.
- Two input dimensions: the network draws a surface $\hat{y} = f_\theta(x_1, x_2)$.
- More hidden neurons provide more adjustable pieces for shaping that curve or surface.

Capacity Is Useful—and Dangerous

Too little capacity

The model cannot follow the main pattern. This is **underfitting**.

Too much effective capacity

The model memorizes training accidents and fails on new data. This is **overfitting**.

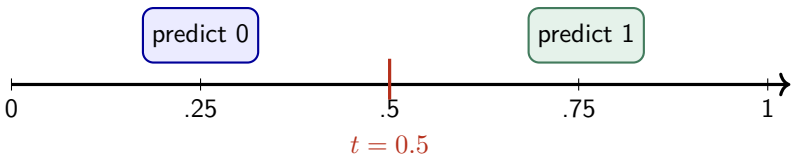
Research habit

Judge architecture choices on validation or test data, not by how perfectly the network fits its training examples.

Evaluate a Binary Classifier

Probability First, Decision Second

$$\hat{p} = P(y = 1 \mid x), \qquad \hat{y} = \begin{cases} 1, & \hat{p} \geq t, \\ 0, & \hat{p} < t. \end{cases}$$



Moving the threshold changes decisions without retraining the network.

A Confusion Matrix Counts Four Outcomes

		Predicted	
		positive	negative
Actual	positive	TP	FN
	negative	FP	TN

- **False positive:** spend resources testing a message that is not useful.
- **False negative:** miss a message that could have been useful.

First identify the real cost

"Positive" is not automatically good, and the two kinds of error rarely cost the same.

Four Common Metrics

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN},$$

$$\text{Precision} = \frac{TP}{TP + FP}$$ Of predicted positives, how many were right?

$$\text{Recall} = \frac{TP}{TP + FN}$$ Of actual positives, how many did we find?

$$F_1 = 2\frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

Hand Check: Ten Message Predictions

$$y = [1, 0, 1, 1, 0, 0, 1, 0, 1, 0]$$

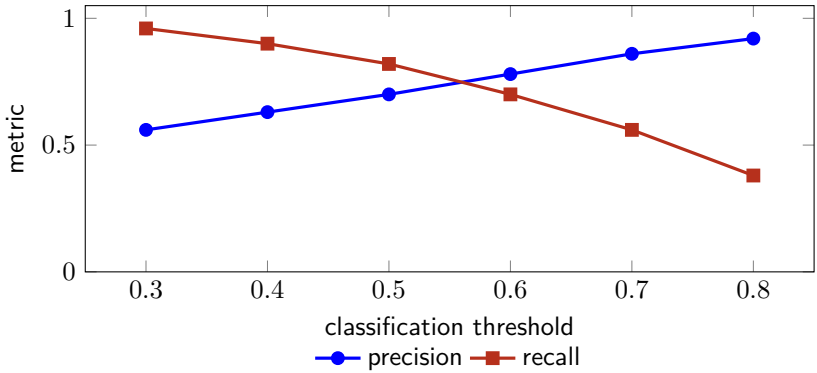
$$\hat{p} = [.91, .72, .64, .42, .38, .81, .55, .20, .49, .10]$$

At $t = 0.50$:

$$\hat{y} = [1, 1, 1, 0, 0, 1, 1, 0, 0, 0].$$

TP	TN	FP	FN	Accuracy	Precision	Recall	F1
3	3	2	2	.60	.60	.60	.60

Thresholds Exchange Precision and Recall



The exact curves depend on the data; the decision principle does not.

Accuracy Can Hide Failure

Suppose only 2 of 100 grid events are dangerous. A model predicts “safe” 100 times.

$$\text{Accuracy} = \frac{98}{100} = 98\%.$$

Yet

$$\text{Recall for danger} = \frac{0}{2} = 0.$$

Always compare against a simple baseline

Class balance, the confusion matrix, and decision costs provide context that accuracy alone cannot.

Loss Is Not the Same as F1

For one binary example, binary cross-entropy is

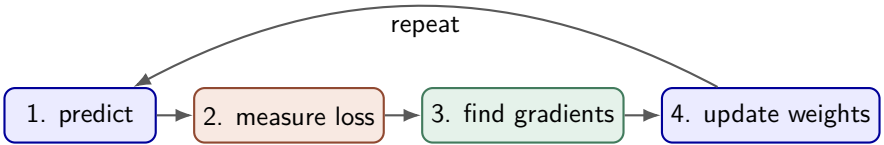
$$L = -\left[y \ln(\hat{p}) + (1-y)\ln(1-\hat{p})\right].$$

Truth	Prediction	Loss	Interpretation
$y = 1$	$\hat{p} = .90$	$-\ln(.90) = .105$	confident and correct
$y = 1$	$\hat{p} = .55$	$-\ln(.55) = .598$	barely correct
$y = 1$	$\hat{p} = .10$	$-\ln(.10) = 2.303$	confident and wrong

The smooth probability-sensitive loss can guide tiny weight changes.

Gradient Descent: Improve the Weights

Training Is Repeated Improvement



One epoch

One epoch means the training procedure has processed every training example once.

A Gradient Is a Local Direction Sign

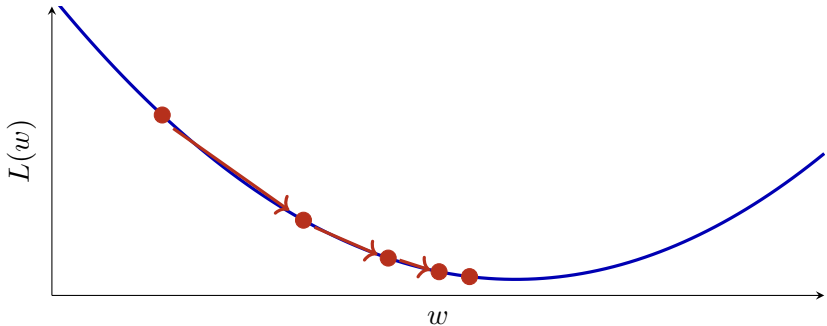
For one parameter w,

$$\frac{\partial L}{\partial w}$$

asks: "If w increases a tiny amount, what happens to the loss?"

- Positive derivative: increasing w raises the loss, so move w down.
- Negative derivative: increasing w lowers the loss, so move w up.
- Near zero: the loss is locally flat in that direction.

Gradient Descent Steps Downhill



The Update Rule

$$\boxed{\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_\theta L}$$

- θ: every weight and bias.
- $\nabla_\theta L$: the collection of all loss derivatives.
- η: the learning rate, or step size.

Too large

Overshoot, bounce, or diverge.

Too small

Improve safely but very slowly.

Why Not Try Every Possible Weight?

A tiny network with 100 parameters already has an enormous search space.

- Testing combinations blindly wastes information from the loss surface.
- A gradient gives a useful local direction for every parameter at once.
- Backpropagation computes those derivatives efficiently by reusing intermediate results.

Analogy

Do not search an entire mountain range square by square. Feel the local slope and take an informed downhill step.

Backpropagation by Hand

Backpropagation Is Organized Chain Rule

If

$$w \longrightarrow z \longrightarrow \hat{p} \longrightarrow L,$$

then

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{p}} \frac{\partial \hat{p}}{\partial z} \frac{\partial z}{\partial w}.$$

Interpretation

Follow every path from a weight to the loss. Multiply local effects along a path; add contributions when several paths meet.

Local Derivatives We Need

$$z = wx + b \qquad \Rightarrow \quad \frac{\partial z}{\partial w} = x, \qquad \frac{\partial z}{\partial b} = 1,$$

$$\mathrm{ReLU}(z) = \max(0, z) \qquad \Rightarrow \quad \mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z < 0, \end{cases}$$

$$\sigma(z) = \frac{1}{1 + e^{-z}} \qquad \Rightarrow \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

At $z = 0$, software chooses a convention for the ReLU derivative, commonly 0.

A Helpful Simplification at the Output

Combine sigmoid output with binary cross-entropy:

$$\hat{p} = \sigma(z^{(2)}), \qquad L = -[y \ln \hat{p} + (1 - y) \ln(1 - \hat{p})].$$

The chain rule simplifies to

$$\boxed{\frac{\partial L}{\partial z^{(2)}} = \hat{p} - y.}$$

The output error signal

If $y = 1$ and $\hat{p} = .63$, then $\delta^{(2)} = .63 - 1 = -.37$. The negative sign says the output score should rise.

The Tiny Network We Will Train

One example:

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \qquad y = 1.$$

Starting parameters:

$$W^{(1)} = \begin{bmatrix} .5 & -.4 \\ .3 & .2 \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} .1 \\ -.1 \end{bmatrix},$$

$$W^{(2)} = \begin{bmatrix} -.7 & 1.2 \end{bmatrix}, \qquad b^{(2)} = -.2.$$

Activation: ReLU in the hidden layer and sigmoid at the output.

Forward Step 1: Hidden Scores

$$z_1^{(1)} = .5(1) - .4(2) + .1 = -.2,$$
$$z_2^{(1)} = .3(1) + .2(2) - .1 = .6.$$

Apply ReLU:

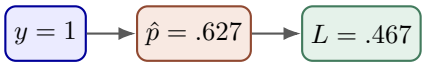
$$h = \mathrm{ReLU}(z^{(1)}) = \begin{bmatrix} 0 \\ .6 \end{bmatrix}.$$

Notice the gate

Hidden neuron 1 is inactive for this example. Its ReLU derivative is 0, so this example sends no gradient through it.

Forward Step 2: Output and Loss

$$z^{(2)} = (-.7)(0) + (1.2)(.6) - .2 = .52,$$
$$\hat{p} = \sigma(.52) \approx .627,$$
$$L = -\ln(.627) \approx .467.$$



Backward Step 1: Output Error

$$\delta^{(2)} = \frac{\partial L}{\partial z^{(2)}} = \hat{p} - y = .627 - 1 = -.373.$$

Since $z^{(2)} = W^{(2)}h + b^{(2)}$,

$$\frac{\partial L}{\partial W^{(2)}} = \delta^{(2)} h^T = -.373 \begin{bmatrix} 0 & .6 \end{bmatrix} = \begin{bmatrix} 0 & -.224 \end{bmatrix},$$

$$\frac{\partial L}{\partial b^{(2)}} = \delta^{(2)} = -.373.$$

Backward Step 2: Send Responsibility to Hidden Neurons

Before the ReLU gate,

$$(W^{(2)})^T \delta^{(2)} = \begin{bmatrix} -.7 \\ 1.2 \end{bmatrix} (-.373) = \begin{bmatrix} .261 \\ -.448 \end{bmatrix}.$$

Apply the ReLU derivatives $[0, 1]^T$:

$$\delta^{(1)} = \left((W^{(2)})^T \delta^{(2)} \right) \odot \mathrm{ReLU}'(z^{(1)}) = \begin{bmatrix} 0 \\ -.448 \end{bmatrix}.$$

Here $\odot$ means element-by-element multiplication.

Backward Step 3: Gradients for the First Layer

Since $z^{(1)} = W^{(1)}x + b^{(1)}$,

$$\frac{\partial L}{\partial W^{(1)}} = \delta^{(1)} x^T = \begin{bmatrix} 0 \\ -.448 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ -.448 & -.896 \end{bmatrix},$$

$$\frac{\partial L}{\partial b^{(1)}} = \delta^{(1)} = \begin{bmatrix} 0 \\ -.448 \end{bmatrix}.$$

Every gradient has the same shape as its parameter.

Update Every Parameter

Use learning rate $\eta = .1$ and $\theta \leftarrow \theta - .1\nabla_\theta L$.

$$W^{(2)}_{\text{new}} = [-.700,\ 1.222], \qquad b^{(2)}_{\text{new}} = -.163,$$

$$W^{(1)}_{\text{new}} = \begin{bmatrix} .500 & -.400 \\ .345 & .290 \end{bmatrix}, \qquad b^{(1)}_{\text{new}} = \begin{bmatrix} .100 \\ -.055 \end{bmatrix}.$$

Subtract the gradient

A negative gradient makes a parameter increase; a positive gradient makes it decrease.

Did the Update Help? Recompute Forward

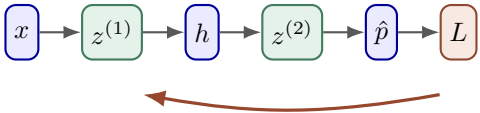
With the updated parameters:

$$
\begin{aligned}
z_{2,\text{new}}^{(1)} &= .345(1) + .290(2) - .055 \approx .869, \\
z_{\text{new}}^{(2)} &= 1.222(.869) - .163 \approx .899, \\
\hat{p}_{\text{new}} &= \sigma(.899) \approx .711, \\
L_{\text{new}} &= -\ln(.711) \approx .341.
\end{aligned}
$$

$$
L: \ .467 \ \longrightarrow \ .341
$$

One step improved this example. Reliable learning requires many examples and repeated evaluation.

The Backward Pass in One Map



then multiply by weights and local derivatives

Backpropagation does not "move data backward"

It moves derivatives—credit or blame for the loss—through the stored forward computation.

From One Example to a Mini-Batch

For B examples, average their losses:

$$L_{\text{batch}} = \frac{1}{B} \sum_{i=1}^{B} L_i.$$

The batch gradient is the average of their gradient contributions.

- Batch size 1: noisy but frequent updates.
- Moderate mini-batch: efficient matrix calculations and smoother direction.
- Full dataset: stable but may be slow and memory-hungry.

Backpropagation in Array Form

```
# Forward
z1 = X @ W1.T + b1
h  = np.maximum(0, z1)
z2 = h @ W2.T + b2
p  = 1 / (1 + np.exp(-z2))

# Backward for sigmoid + binary cross-entropy
delta2 = (p - y) / len(X)
dW2 = delta2.T @ h
db2 = delta2.sum(axis=0)
delta1 = (delta2 @ W2) * (z1 > 0)
dW1 = delta1.T @ X
db1 = delta1.sum(axis=0)

# Gradient-descent update
W1 -= learning_rate * dW1
b1 -= learning_rate * db1
W2 -= learning_rate * dW2
b2 -= learning_rate * db2
```

How Can We Check a Hand-Derived Gradient?

Nudge one parameter by a tiny ε:

$$\frac{\partial L}{\partial w} \approx \frac{L(w+\varepsilon) - L(w-\varepsilon)}{2\varepsilon}.$$

This **finite-difference gradient check** is slow but valuable for debugging.

Use it as a test, not a training algorithm

Backpropagation gets all gradients in roughly the cost of a small number of forward passes; finite differences need extra forward passes for every parameter.

Why GPUs Make Neural Networks Fast

Most Neural-Network Work Is Matrix Arithmetic

A dense layer for a batch is

$$Z = XW^T + b.$$

Each output cell is an independent dot product:

$$Z_{ij} = \sum_k X_{ik} W_{jk} + b_j.$$

Opportunity

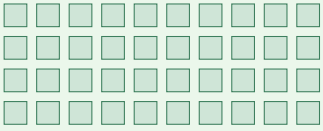
Thousands of output cells can be calculated at the same time. This repeated, regular arithmetic is exactly the pattern GPUs are designed to accelerate.

CPU and GPU: Different Strengths

CPU: a few powerful workers

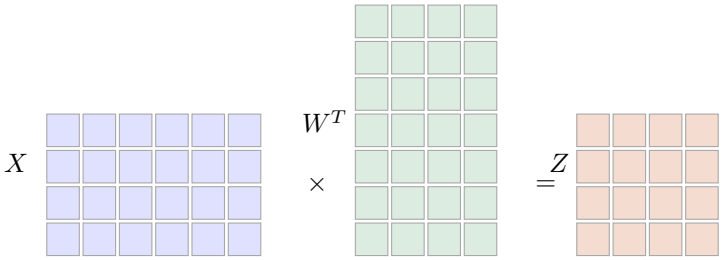
- strong general-purpose cores;
- excellent for branching and varied tasks;
- low-latency control work.

GPU: many arithmetic workers



- many smaller parallel units;
- high arithmetic throughput;
- thrives on large regular batches.

Visualizing One Matrix Multiplication



Mostly sequential picture

A CPU assigns a smaller number of strong workers to the output cells.

Massively parallel picture

A GPU schedules many output cells and multiply-add operations together.

Why Small Networks May Not Benefit

Sending data to a GPU and launching GPU work has overhead.

- Tiny model + one example: CPU may finish before GPU setup pays off.
- Large matrices + large batch: parallel work dominates overhead.
- Data should stay on the GPU across many operations; repeated transfers are expensive.

GPU does not make every line of Python faster

Speed comes from expressing large compatible operations as GPU kernels, usually through libraries such as PyTorch.

Inference Memory: What Must Stay Available?

During inference, memory commonly holds:

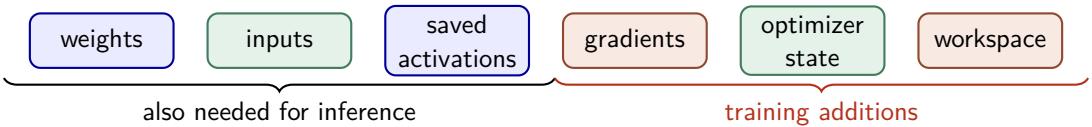
Earlier activations can often be released as soon as later layers no longer need them.

Main controls

Parameter count, numeric precision, batch size, sequence/image size, and implementation details.

Training Memory: Keep the Evidence for Backward

Training commonly needs all inference items plus:



Key reason

Backpropagation needs intermediate forward activations to calculate local derivatives. Training also stores a gradient for each parameter and often extra optimizer statistics.

A Rough Memory Accounting Example

Suppose a model has P parameters stored in 32-bit floating point: about $4P$ bytes.

Stored item	Inference	Training with Adam-like optimizer
weights	$4P$	$4P$
parameter gradients	—	$4P$
two optimizer states	—	about $8P$
activations/workspace	yes	usually much more, because saved
parameter-related subtotal	about $4P$	about $16P$ before extras

Real totals vary with optimizer, mixed precision, master weights, quantization, checkpointing, architecture, batch shape, and framework workspace.

How Training Fits When Memory Is Tight

- Reduce batch size or input resolution/sequence length.
- Use lower-precision formats when supported.
- Use gradient accumulation to imitate a larger batch across several smaller batches.
- Recompute some activations during backward instead of storing all of them: **activation checkpointing**.
- Freeze most parameters or use parameter-efficient fine-tuning when appropriate.

Tradeoff

Saving memory often costs additional computation, lower throughput, or a change in optimization behavior.

Interactive Demonstration and Wrap-Up

Demo: Manually Shape a Neural-Network Function

Configure:

- one or two input dimensions;
- one to three hidden layers;
- the number of neurons in each hidden layer;
- every weight and bias; and
- an exemplar set of data points.

Then display the network's curve or 3D surface and the current mean loss.

Student challenge

Without automatic training, adjust parameters until the network function approaches the points. Which changes move the whole function? Which create a bend?

Demo Discussion Questions

1. What happens when an output-layer weight changes?
2. What happens when a hidden bias moves a ReLU's activation boundary?
3. Does adding neurons automatically lower the loss?
4. Can two different parameter settings draw similar functions?
5. Why would adjusting hundreds of parameters manually be impractical?
6. How does backpropagation replace guessing with a calculated direction?

Common Misconceptions

- **"Backprop is the update."** Backprop computes gradients; the optimizer uses them to update parameters.
- **"A lower training loss proves a better model."** Generalization must be checked on unseen data.
- **"The threshold trains the network."** The threshold turns probabilities into decisions after the forward pass.
- **"GPU memory is only model weights."** Activations, gradients, optimizer state, inputs, and workspaces also matter.
- **"GPU is always faster."** Parallel workload must be large enough to repay overhead.

Exit Ticket

Answer each in one sentence or one calculation:

1. If $TP = 8$, $FP = 2$, and $FN = 4$, calculate precision and recall.
2. Why can F1 be more informative than accuracy on imbalanced data?
3. If $y = 1$ and $\hat{p} = .7$, what is $\delta^{(2)}$ for sigmoid plus binary cross-entropy?
4. What does the ReLU derivative do to a negative hidden score?
5. Why do we subtract the gradient?
6. Name two items training stores that ordinary inference does not.
7. Why is matrix multiplication suitable for a GPU?

Homework for This Week

Manual derivation

Complete a full forward pass, binary cross-entropy loss, backward pass, and one gradient-descent update for a two-input MLP.

Manual programming

Implement forward propagation, confusion-matrix metrics, backpropagation, and a training loop using NumPy only—no autograd for the core task.

GPU explanation

Draw or create a diagram explaining parallel matrix arithmetic and why training memory exceeds inference memory.

Key Takeaways

1. An MLP composes weighted sums and nonlinear activations to approximate a mapping.
2. Metrics evaluate decisions; a smooth loss trains probabilities.
3. Gradient descent updates parameters opposite the loss gradient.
4. Backpropagation applies the chain rule efficiently from loss to earlier layers.
5. GPUs accelerate the large, regular matrix operations used in forward and backward passes.
6. Training needs extra memory for saved activations, gradients, and optimizer state.